



**SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES**



ASSESSMENT TOOL 2

Case Study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1104

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. **(BL4)**

Assessment Tool Weightage: **40%**

QUESTIONS: Any 4

Total Marks: 40

Code of Conduct:

I **S.Poojitha/ 192320076** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Poojitha

Reg. No: 192320076

Note: The assessment sheet lists 6 case studies and instructs students to answer "Any 4." The four attempted in this submission are: (1) Online Hotel Reservation System, (3) Online Retail Shopping Platform, (4) Smart Waste Management System, and (5) Smart Healthcare Management System.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1: Online Hotel Reservation System

Scenario: A travel company plans to develop an online hotel reservation platform where customers can search hotels, book rooms, make payments, and provide reviews. Hotel managers can update room availability and pricing, while administrators manage users and transactions.

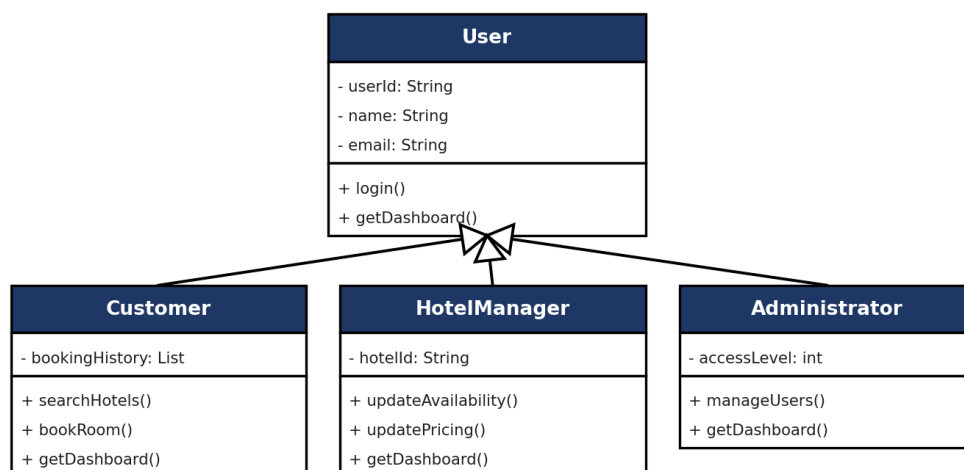
Question: Analyze the system and explain how object-oriented principles (encapsulation, inheritance, polymorphism) can be applied. Also, suggest a suitable SDLC model and justify your choice for this system. (10 Marks)

1. Application of Object-Oriented Principles

The hotel reservation platform involves three distinct user roles (Customer, Hotel Manager, Administrator) and several interacting entities (rooms, bookings, payments), making it a natural fit for object-oriented analysis:

- **Encapsulation:** The Booking class hides its internal state transitions (pending → confirmed → cancelled) behind controlled methods rather than exposing a raw status field, and the Payment class encapsulates the details of gateway integration so that other classes only interact with a simple processPayment() method without needing to know how payment is actually processed.
- **Inheritance:** A common User base class defines shared attributes (userId, name, email) and behaviour (login()), while Customer, HotelManager, and Administrator each inherit from it and extend it with role-specific attributes and methods, avoiding duplicated code across the three roles.
- **Polymorphism:** Each subclass overrides getDashboard() to return content appropriate to its role — a Customer sees their bookings, a HotelManager sees room-availability controls, an Administrator sees user/transaction management — so calling the same method name on any User object produces the correct role-specific behaviour automatically.

Hotel Reservation System — User Inheritance Hierarchy



Each subclass overrides getDashboard() to return role-specific content — an example of polymorphism. Attributes such as bookingHistory and hotelId are kept private (encapsulated) within their own classes.

Figure 1.1 — Illustrative User inheritance hierarchy for the Hotel Reservation System

2. Recommended SDLC Model

Recommended Model: **Incremental Model**

The core booking requirements (search, book, pay) are reasonably well understood upfront, since online hotel booking is a mature, well-established domain — but the platform will still benefit from real user feedback on secondary features such as reviews, dynamic pricing, and personalised recommendations. The Incremental Model allows the team to deliver the essential search-book-pay flow as the first working increment, so the business can start generating value and gathering real usage data quickly, while later increments add the review system, manager-side pricing tools, and administrative reporting. This reduces risk compared to a pure Waterfall approach (where all features would need to be fully specified before any code is delivered) while still providing more structure and predictability than a fully exploratory Agile process, which fits well since the core domain is not highly uncertain.

Case Study 3: Online Retail Shopping Platform

Scenario: A retail company wants to develop an e-commerce platform where customers can browse products, place orders, track deliveries, and make online payments. Vendors manage product catalogs, while administrators monitor transactions and inventory.

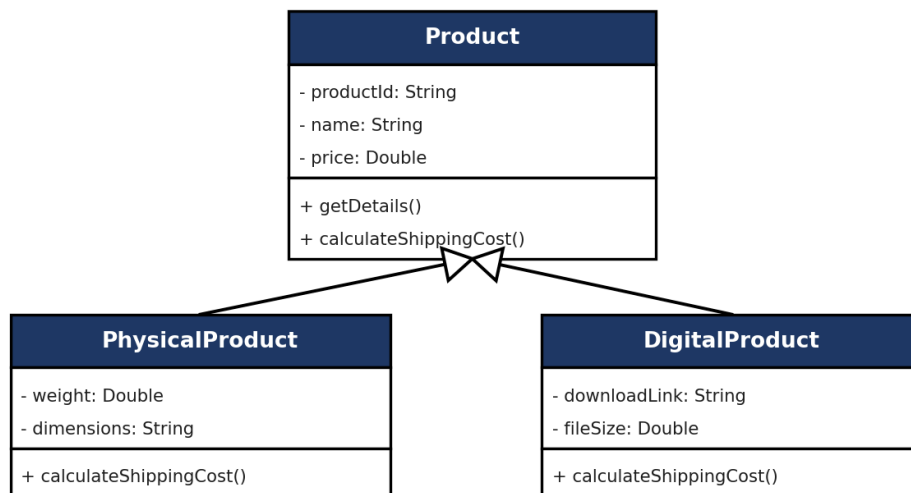
Question: Discuss how modular software design can be achieved using object-oriented techniques. Evaluate different SDLC models and select the most appropriate one for this scenario with justification. (10 Marks)

1. Application of Object-Oriented Principles

An e-commerce platform naturally decomposes into several interacting concerns — product catalog, orders, payments, inventory — which object-oriented design captures directly as classes with clear responsibilities:

- **Encapsulation:** The Order class encapsulates its own state machine (placed → shipped → delivered), only allowing transitions through defined methods such as markShipped(), so other parts of the system cannot corrupt an order's status directly; similarly, Inventory encapsulates stock-adjustment logic behind an updateStock() method.
- **Inheritance:** A base Product class defines shared attributes (productId, name, price) and behaviour, while PhysicalProduct and DigitalProduct subclasses extend it with type-specific attributes (weight/dimensions vs downloadLink), avoiding duplicated logic across product types.
- **Polymorphism:** calculateShippingCost() is overridden differently in each Product subclass — physical products compute a courier-based cost while digital products return zero — so the checkout logic can call the same method on any Product object without needing to know its concrete type.

Retail Platform — Product Inheritance Hierarchy



calculateShippingCost() is overridden in each subclass (physical items incur courier cost; digital items return zero) — polymorphism through method overriding.

Figure 3.1 — Illustrative Product inheritance hierarchy for the Retail Platform

2. Modular Design Using Object-Oriented Techniques

Modularity is achieved by separating the system into loosely-coupled classes/modules — User Management, Catalog Management, Order Processing, Payment Processing, and Inventory Management — each communicating with the others only through well-defined method interfaces rather than sharing internal data directly. This means a new payment gateway or a new product category can be added by extending or introducing a class (e.g., a new Product subclass or a new PaymentMethod implementation) without modifying the order-processing or catalog logic, following the open/closed principle. It also allows different modules to be developed, tested, and deployed by different teams in parallel, and lets faults in one module (e.g., a payment gateway outage) be isolated without necessarily breaking browsing or order-tracking functionality.

3. Recommended SDLC Model

Recommended Model: Agile (Scrum) Model

Among the SDLC models considered — Waterfall, Incremental, Spiral, and Agile — Agile is the strongest fit here. E-commerce platforms operate in a fast-moving, competitive environment where requirements evolve continuously (new promotional features, changing vendor needs, seasonal catalog changes, shifting customer expectations), which is poorly suited to Waterfall's assumption of fixed upfront requirements. Agile's short sprints allow the team to release a minimum viable storefront quickly, then continuously prioritise and integrate feedback from customers and vendors sprint by sprint, which is essential in retail where competitor pressure and customer behaviour data should directly influence what gets built next. Spiral was also considered, since it explicitly manages risk, but its heavier process overhead is better suited to high-risk, safety-critical systems than to a

commercially-driven platform where speed of iteration matters more than formal risk analysis at every stage.

Case Study 4: Smart Waste Management System

Scenario: A municipality introduces a smart waste management system where IoT-enabled bins monitor waste levels and automatically notify collection vehicles when bins are full. Citizens can also report waste-related issues through a mobile application.

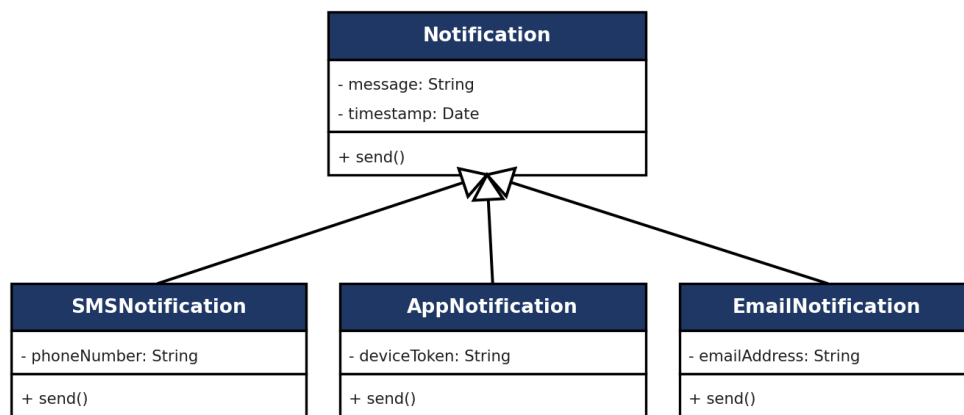
Question: Apply object-oriented system development principles to design this system. Identify the life cycle model best suited for handling real-time data, sensor integration, and evolving requirements, and explain why. (10 Marks)

1. Application of Object-Oriented Principles

This system combines IoT hardware (sensors, connectivity), real-time data processing, and a citizen-facing application, which object-oriented design handles by cleanly separating sensing, notification, and reporting concerns:

- **Encapsulation:** The Bin class encapsulates its raw fill-level sensor readings and internal threshold logic, exposing only a simple `isFull()` or `getStatus()` method to the rest of the system, so collection-scheduling logic never needs to interpret raw sensor values directly.
- **Inheritance:** A base Notification class defines shared attributes (message, timestamp) and a `send()` method, while `SMSNotification`, `AppNotification`, and `EmailNotification` subclasses extend it with channel-specific attributes (phoneNumber, deviceToken, emailAddress), avoiding duplicated notification-dispatch logic.
- **Polymorphism:** Because `send()` is overridden per channel, the bin-monitoring logic can call `notify.send()` on any Notification object — whether it resolves to an SMS, a push notification, or an email — without needing separate code paths for each channel.

Smart Waste Management — Notification Inheritance Hierarchy



send() is overridden per channel, so BinMonitor can call `notify.send()` on any Notification object without knowing which channel is used underneath — polymorphism.

Figure 4.1 — Illustrative Notification inheritance hierarchy for the Smart Waste Management System

2. Recommended SDLC Model

Recommended Model: **Spiral Model**

This system carries substantial technical risk and uncertainty that Waterfall or a simple Incremental approach would handle poorly: sensor hardware reliability, field connectivity issues, calibration of fill-level thresholds, and integration between IoT devices and backend software are all areas where problems are likely to surface only through hands-on experimentation, not upfront specification. The Spiral Model's defining feature — repeating a plan → identify-and-analyse-risk → engineer-and-test → evaluate cycle — is specifically designed for exactly this kind of high-uncertainty, hardware-integrated project: each cycle can prototype and test a risky area (e.g., sensor-to-cloud connectivity under real municipal conditions) before committing further budget and effort to it. Its iterative nature also naturally accommodates evolving requirements, such as citizens requesting new issue-reporting categories or the municipality adding new sensor types over time, which a rigid sequential model would struggle to absorb without a costly restart.

Case Study 5: Smart Healthcare Management System

Scenario: A multi-specialty hospital plans to develop a Smart Healthcare Management System where patients can book appointments, consult doctors, access medical records, and make online payments. Doctors update patient diagnoses and prescriptions, while hospital administrators manage departments, staff, and billing information.

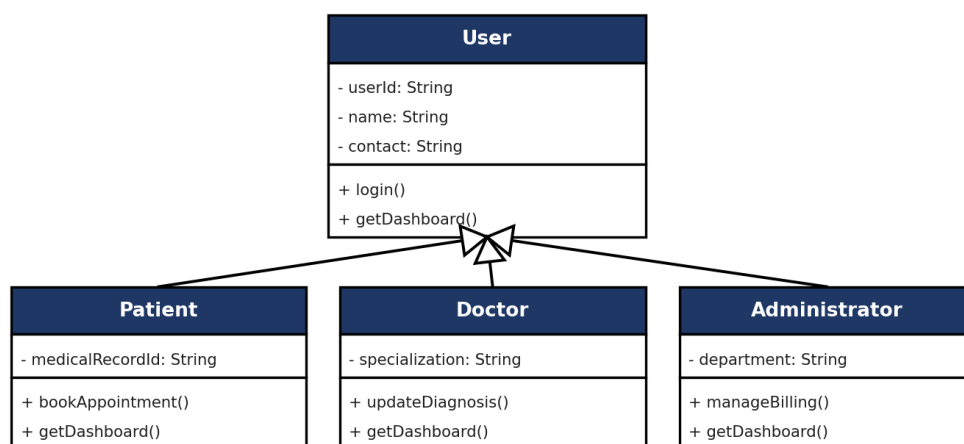
Question: Analyze the system and explain how object-oriented principles (encapsulation, inheritance, and polymorphism) can be applied. Also, recommend a suitable SDLC model for developing this healthcare system and justify your choice. (10 Marks)

1. Application of Object-Oriented Principles

A hospital management system handles sensitive patient data and multiple professional roles, making disciplined object-oriented design especially important for both correctness and privacy:

- **Encapsulation:** The MedicalRecord class keeps sensitive patient data private and only accessible through controlled, access-checked methods (e.g., viewRecord() callable only by the owning Patient or an authorized Doctor), rather than exposing raw fields — directly supporting the confidentiality that healthcare data protection requires.
- **Inheritance:** A base User class defines shared attributes (userId, name, contact) and behaviour (login()), while Patient, Doctor, and Administrator subclasses extend it with role-specific attributes and methods (medicalRecordId for Patient, specialization for Doctor, department for Administrator), avoiding duplicated authentication and profile logic.
- **Polymorphism:** getDashboard() is overridden per role so that the same method call surfaces appointment-booking for a Patient, diagnosis-entry tools for a Doctor, and billing/department management for an Administrator, letting the rest of the system treat all logged-in users uniformly as User objects while still getting correct role-specific behaviour.

Smart Healthcare System — User Inheritance Hierarchy



MedicalRecordId is private to Patient and only exposed through controlled methods (encapsulation), protecting sensitive health data while getDashboard() is polymorphic per role.

Figure 5.1 — Illustrative User inheritance hierarchy for the Smart Healthcare System

2. Recommended SDLC Model

Recommended Model: V-Model (Verification and Validation Model)

Healthcare systems carry a high cost of failure — incorrect diagnosis records, billing errors, or data-privacy breaches can directly harm patients or violate regulatory requirements — so rigorous, systematic testing at every stage of development is essential rather than optional. The V-Model pairs each development phase with a corresponding, pre-planned testing phase (requirements ↔ acceptance testing, system design ↔ system testing, module design ↔ unit testing), ensuring that verification and validation activities are designed from the very start of the project rather than added on afterward. This traceability between requirements and tests is particularly valuable for demonstrating regulatory compliance and patient-safety assurance. The V-Model does assume relatively well-understood requirements upfront, which is a reasonable assumption here since core hospital workflows — appointment booking, diagnosis recording, billing — are well-established administrative and clinical processes, unlike the more exploratory, hardware-dependent uncertainty seen in the Smart Waste Management case.